ENGENHARIA DE SOFTWARE · 2026.2

PATINHAS

Modelagem e Implementação

Sistema com Microserviços e Componentes de Software

Catálogo demonstrativo e recomendação explicável de pets

Identificação acadêmica — preencher antes da entrega

Discente: preencher antes da entrega

Instituição / curso: preencher antes da entrega

Docente: preencher antes da entrega

Repositório GitHub: inserir o link confirmado antes da entrega

Entrega prevista: 9 de novembro de 2026

Versão de trabalho em 27 de outubro de 2026

Sumário

1 Enunciado e objetivo 1
2 Metodologia ágil 1
2.1 Product Backlog 1
2.2 Sprint Backlog 2
2.3 Técnicas de reuso de software 2
3 Arquitetura do sistema 2
3.1 Visão geral 2
3.2 API Gateway 3
3.3 Especificação dos microserviços 3
3.3.1 Microserviço Pets 3
3.3.2 Microserviço Preferências/Favoritos 4
3.3.3 Microserviço Recomendações 4
3.4 Componentes reutilizáveis 4
3.5 Sistema de recomendação 5
3.5.1 Sequência de favoritos 7
3.6 Tela de estado dos serviços 7
4 Tecnologias utilizadas 7
4.1 Endpoints acessíveis 8
5 Funcionamento passo a passo 8
5.1 Página inicial e transparência 8
5.2 Questionário 9
5.3 Recomendações 9
5.4 Catálogo, filtros e perfil 10
5.5 Favoritos 12
5.6 Estado e documentação 12
6 Organização, testes e estado atual 13
6.1 Limites e melhorias futuras 13
7 Reuso demonstrado e referências visuais 13
8 Glossário 14
9 Referências 14
10 Conferência para a entrega final 14

1 Enunciado e objetivo

O projeto **Patinhas** modela e implementa uma aplicação web acadêmica para explorar perfis demonstrativos de cães e gatos, guardar favoritos e estimar compatibilidade entre um perfil de preferência e os pets cadastrados. A proposta atende ao tema "Sistema usando Microservices e Componentes de Software [e AgentesAI]": a parte entre colchetes é opcional, portanto esta primeira versão não usa agentes nem IA generativa.

Todos os nomes, imagens e atributos apresentados são fictícios. O produto não publica anúncios reais, não representa abrigos e não substitui uma avaliação responsável de adoção. O objetivo é demonstrar conceitos de Engenharia de Software com escopo controlado, execução local e uma interface que deixe os limites claros.

O percurso implementado é: explorar catálogo, aplicar filtros, abrir detalhes, preencher questionário, receber recomendações justificadas e marcar ou remover favoritos. A interface é servida pelo Gateway, enquanto as APIs e bancos pertencem aos serviços responsáveis.

2 Metodologia ágil

Foi escolhida uma adaptação leve de Scrum (Scrum). O Product Backlog ordena os resultados necessários; Sprint Backlogs agrupam trabalho por incrementos demonstráveis. O prazo informado é 9 de novembro de 2026. Como o protótipo já teve uma primeira implementação em 27 de outubro, o documento distingue o que está implementado do que ainda é atividade planejada. Não afirma que cerimônias de equipe não realizadas ocorreram.

2.1 Product Backlog

Os itens estão ordenados por dependência e valor de demonstração:

1. **PB1 — Catálogo e transparência:** apresentar pets fictícios e comparar suas características, com aviso explícito de demonstração.
2. **PB2 — Busca:** filtrar por espécie, porte, idade e energia.
3. **PB3 — Perfil:** abrir detalhes, história e condições de convivência.
4. **PB4 — Preferências:** responder ao questionário sem cadastro e persistir as escolhas demonstrativas.
5. **PB5 — Recomendação:** ordenar os pets e explicar a pontuação por critérios compreensíveis.
6. **PB6 — Favoritos:** adicionar, consultar e remover pets com persistência SQLite.
7. **PB7 — Microserviços e Gateway:** manter serviços independentes, APIs, health checks e ponto único de entrada.
8. **PB8 — Reuso de interface:** reutilizar cartão de pet e indicador de compatibilidade.
9. **PB9 — Documentação técnica:** OpenAPI/Swagger, arquitetura, regra de recomendação e instruções de execução.
10. **PB10 — Relatório e demonstração:** incluir telas, tecnologia, explicações passo a passo e referências.
11. **PB11 — Distribuição:** guardar o código em repositório GitHub e incluir seu endereço na versão final da capa.

2.2 Sprint Backlog

Sprint e período	Itens	Objetivo / situação
1 · 27–29 out.	PB1, PB7, PB8, PB9	Recortar o problema, modelar serviços e estabelecer a estrutura. Primeira versão funcional iniciada em 27/10; a documentação da modelagem continua aberta.
2 · 30 out.–2 nov.	PB1–PB5	Consolidar catálogo, filtros, perfis, questionário e recomendações explicáveis; revisar os casos menos compatíveis.
3 · 3–5 nov.	PB6–PB9	Validar persistência, reuso de componentes, health checks, documentação das APIs e adaptação para celular.
4 · 6–9 nov.	PB9–PB11	Executar testes, finalizar relatório, preencher identificação e preparar a entrega GitHub até 9/11.

2.3 Técnicas de reuso de software

A reutilização ocorre em níveis distintos. A decomposição por microserviços permite que cada responsabilidade seja consumida por mais de um fluxo da aplicação. Na interface, `PetCard` e `MatchScore` são Web Components reutilizados no catálogo, no mural de favoritos e nos resultados de recomendação. As APIs são declaradas uma vez em FastAPI e descritas por OpenAPI (OpenAPI), com Swagger UI gerada automaticamente. FastAPI documenta esse fluxo automático na sua documentação oficial [2].

Também foram usadas bibliotecas externas FastAPI, Uvicorn e HTTPX, declaradas em `requirements.txt`, e APIs da plataforma Web. Essa dependência de bibliotecas é reuso de software, mas não significa que o protótipo reutilize a identidade, o código ou as imagens dos sites visuais consultados.

3 Arquitetura do sistema

3.1 Visão geral

A pessoa usuária acessa uma interface HTML/CSS/JavaScript. O navegador chama apenas o API Gateway na porta 3000. O Gateway serve os arquivos estáticos, encaminha as rotas de domínio e verifica a saúde dos três serviços. O serviço de Recomendações consulta Pets e Preferências por HTTP interno; não lê diretamente os arquivos SQLite de outros serviços.

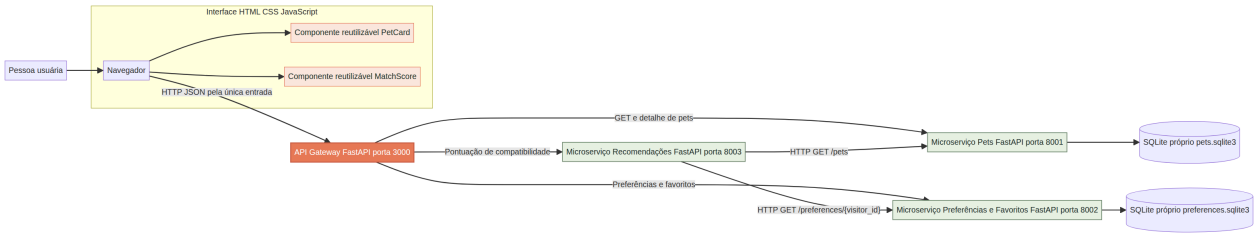


Figura 1 — Arquitetura e propriedade dos bancos de dados.

Os processos locais são iniciados por `run.py`: Gateway na porta 3000; Pets em 8001; Preferências/Favoritos em 8002; Recomendações em 8003. Pets é dono de `data/pets.sqlite3`; Preferências/Favoritos é dono de `data/preferences.sqlite3`. Recomendações não exige banco próprio.

3.2 API Gateway

O Gateway centraliza o acesso da interface sem fundir as regras de domínio. O método auxiliar `forward` faz a chamada HTTP ao endereço interno, preserva o status da resposta e converte uma falha de rede em HTTP 503. Abaixo está um trecho representativo do arquivo `gateway/main.py`:

```
async def forward(service: str, path: str, method: str = "GET", params=None,
body=None):
    url = f"{SERVICES[service]}{path}"
    try:
        async with httpx.AsyncClient(timeout=4.0) as client:
            response = await client.request(
                method, url, params=params, json=body
            )
        try:
            payload = response.json()
        except ValueError:
            payload = {"detail": "O serviço retornou uma resposta inválida."}
        return JSONResponse(status_code=response.status_code, content=payload)
    except httpx.RequestError:
        return JSONResponse(
            status_code=503,
            content={"detail": f"O microserviço {service} está indisponível no
momento."},
        )
```

Por exemplo, `GET /api/pets` encaminha filtros para Pets; `PUT /api/preferences` persiste o questionário via Preferências; e `GET /api/recommendations` consulta o serviço de recomendação. `GET /api/status` chama `/health` dos serviços e informa `online` ou `offline`. O Gateway fornece seu próprio OpenAPI/Swagger em `/docs`.

3.3 Especificação dos microserviços

3.3.1 Microserviço Pets

Responsabilidade: ser a única fonte de verdade do catálogo fictício. Inicializa seu arquivo SQLite se necessário, insere três registros de demonstração e valida os filtros. Rotas: `GET /health`, `GET /pets` com `species`, `size`, `age`, `energy` e `limit`, além de `GET /pets/{pet_id}`. Acesso ao código: `services/pets/main.py`.

O endpoint de detalhe consulta somente o banco que pertence a Pets; se o ID não existir, responde HTTP 404:

```
@app.get("/pets/{pet_id}", tags=["Pets"])
def get_pet(pet_id: int) -> dict:
    with connect() as db:
        row = db.execute("SELECT * FROM pets WHERE id = ?", (pet_id,)).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="Pet não encontrado no catálogo
de demonstração.")
    return serialize(row)
```

3.3.2 Microserviço Preferências/Favoritos

Responsabilidade: guardar as respostas do quiz e os IDs favoritos no SQLite próprio. Rotas: `GET/PUT /preferences/{visitor_id}`, `GET /favorites/{visitor_id}`, `PUT /favorites/{visitor_id}/{pet_id}` e `DELETE /favorites/{visitor_id}/{pet_id}`. O protótipo usa o identificador `demo`, sem autenticação; logo, não promete separar visitantes reais. Acesso ao código: `services/preferences/main.py`.
Trecho documentado de persistência idempotente do favorito:

```
@app.put("/favorites/{visitor_id}/{pet_id}")
def add_favorite(visitor_id: str, pet_id: int):
    with connect() as db:
        db.execute(
            "INSERT OR IGNORE INTO favorites (visitor_id, pet_id) VALUES (?, ?)",
            (visitor_id, pet_id),
        )
    return {"visitor_id": visitor_id, "pet_id": pet_id, "saved": True}
```

3.3.3 Microserviço Recomendações

Responsabilidade: consultar pets e preferências por HTTP, pontuar cada par de dados, ordenar do mais compatível para o menos compatível e devolver explicações legíveis. Rotas: `GET /health` e `GET /recommendations/{visitor_id}`. Não possui banco; seus dados de entrada pertencem aos outros serviços. Acesso ao código: `services/recommendations/main.py`.

3.4 Componentes reutilizáveis

`PetCard` é um elemento personalizado registrado como `<pet-card>`. Renderiza imagem, espécie, idade, nome, temperamento, porte, energia e ações para detalhes/favorito. Dispara eventos DOM (`pet-details` e `favorite-toggle`), mantendo o componente separado da lógica que chama APIs. O mesmo cartão é criado para o catálogo, as recomendações e os favoritos.
`MatchScore` é o elemento `<match-score>`, usado dentro dos cartões recomendados. Mostra percentual, rótulo e justificativas em texto; o significado não depende apenas da cor. MDN descreve a API de elementos personalizados usada para esse padrão [4]. Código: `web/js/components.js`.

```
this.querySelector(".favorite-toggle").addEventListener("click", () => {
  this.dispatchEvent(new CustomEvent("favorite-toggle", {
```

```
    bubbles: true,
    composed: true,
    detail: { petId: Number(petId), currentlyFavorite: favorite },
  }));
});
this.querySelector(".details-button").addEventListener("click", () => {
  this.dispatchEvent(new CustomEvent("pet-details", {
    bubbles: true,
    composed: true,
    detail: { petId: Number(petId) },
  }));
});

if (!customElements.get("match-score")) customElements.define("match-score",
MatchScore);
if (!customElements.get("pet-card")) customElements.define("pet-card", PetCard);
```

Este trecho real de `web/js/components.js` emite eventos reutilizáveis para a camada de interface; as classes completas incluem a marcação, atributos acessíveis e fallback de imagem.
O indicador de compatibilidade é outro elemento personalizado, `MatchScore`. Ele transforma os atributos recebidos em marcação textual reutilizável:

```
class MatchScore extends HTMLElement {
  connectedCallback() {
    const score = escapeHTML(this.getAttribute("score") || "0");
    const label = escapeHTML(this.getAttribute("label") || "Compatibilidade");
    const reasons = escapeHTML(this.getAttribute("reasons") || "");
    this.innerHTML = `
      <div class="match-score-inner">
        <div class="match-score-top">
          <span class="match-score-value">${score}% combina</span>
          <span class="match-score-label">${label}</span>
        </div>
        ${reasons ? `<p class="match-reasons">${reasons}</p>` : ""}
      </div>`;
  }
}
```

3.5 Sistema de recomendação

O recomendador é determinístico, explicável e local; não usa um serviço de IA. Cada pet recebe até 100 pontos: espécie (25), porte (20), atividade (até 30), tipo de moradia (15) e convivência com crianças quando informada (10). Preferências “qualquer espécie” e “qualquer porte” recebem os pontos integrais de seus

critérios. A atividade exata vale 30; diferença de um nível vale 19; diferença de dois vale 8. O rótulo é "Ótima combinação" a partir de 80, "Pode combinar" a partir de 60 e "Vale conhecer melhor" abaixo disso. Trecho da função em `services/recommendations/main.py`:

```
activity_points = {
    "low": {"low": 30, "medium": 19, "high": 8},
    "medium": {"low": 19, "medium": 30, "high": 19},
    "high": {"low": 8, "medium": 19, "high": 30},
}
score += activity_points[preferred_activity][pet["energy_level"]]
if preferred_activity == pet["energy_level"]:
    reasons.append("O nível de energia combina com a sua rotina.")
else:
    reasons.append("Considere se o nível de energia combina com a sua rotina.")
```

A sequência de chamadas para calcular um resultado é mostrada na figura seguinte:

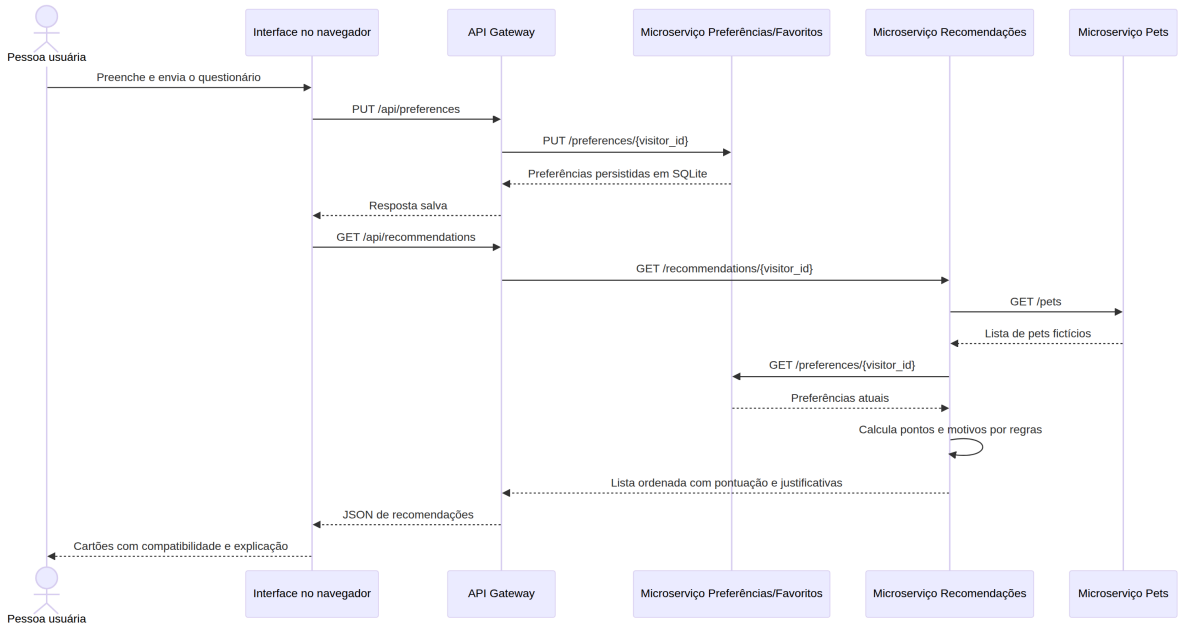


Figura 2 — O quiz salva preferências e a recomendação consulta dados pelos contratos HTTP.

3.5.1 Sequência de favoritos

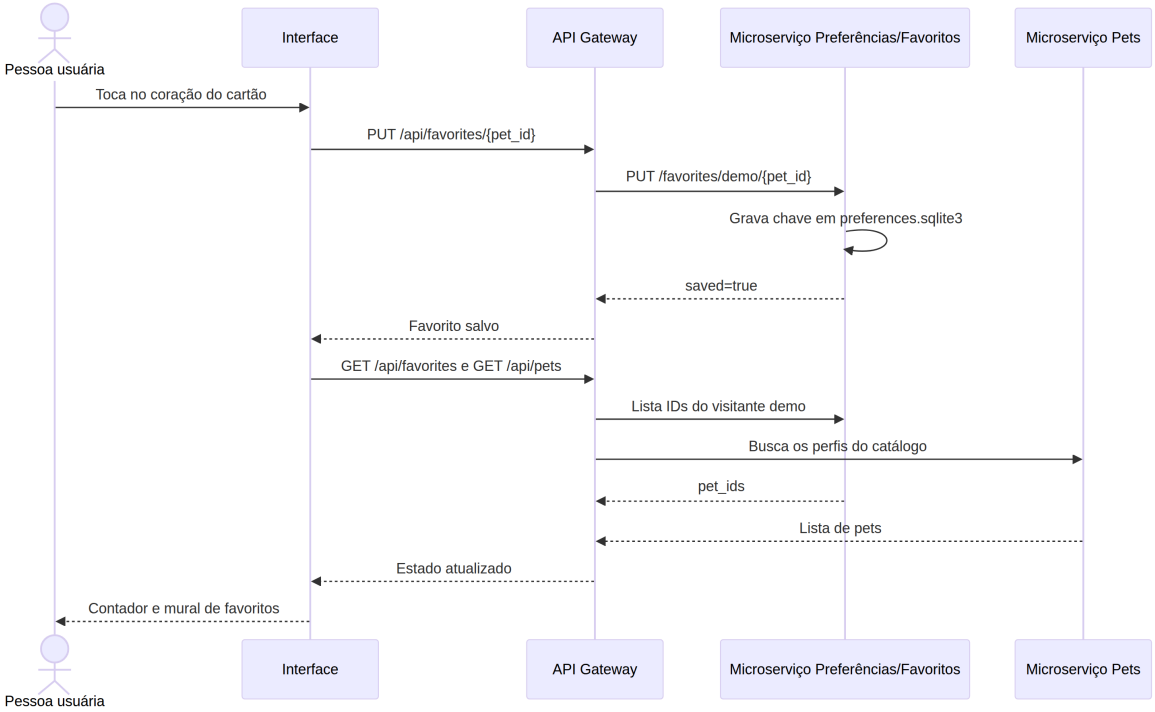


Figura 3 — O serviço de Preferências/Favoritos persiste a escolha, enquanto o Gateway recompõe a tela usando as APIs proprietárias.

3.6 Tela de estado dos serviços

O painel “Por trás das Patinhas” exibe o Gateway e os três microserviços com seus health checks. Cada serviço tem um link para a documentação interativa que o Gateway encaminha. A Figura 8 mostra a listagem de serviços em execução; o painel fica na parte inferior da página.

4 Tecnologias utilizadas

Parte	Linguagem / framework	Banco	Uso
API Gateway	Python 3.12.3, FastAPI 0.162.0	Nenhum	Servir a interface, encaminhar APIs, estado e OpenAPI.
Pets	Python 3.12.3, FastAPI 0.162.0	SQLite próprio	Catálogo, filtros, detalhes e health check.
Preferências/Favoritos	Python 3.12.3, FastAPI 0.162.0	SQLite próprio	Questionário, inclusão, consulta e remoção de favoritos.
Recomendações	Python 3.12.3, FastAPI 0.162.0	Nenhum	Integra APIs internas e calcula pontuação explicável.

Interface	HTML, CSS, JavaScript e Web Components	—	Catálogo responsivo, quiz, detalhes, favoritos e componentes reutilizáveis.

Uvicorn 0.54.0 executa as aplicações ASGI; HTTPX 0.28.1 faz chamadas entre serviços. A biblioteca padrão `sqlite3` abre os arquivos de banco locais. `requirements.txt` fixa as versões utilizadas — FastAPI 0.162.0, Uvicorn 0.54.0 e HTTPX 0.28.1 — para tornar a instalação demonstrativa reproduzível. A documentação oficial de Python e SQLite descreve suas bibliotecas e mecanismos [1] [3].

4.1 Endpoints acessíveis

Gateway / rota	Operação	Destino ou efeito
`GET /api/pets`	Listar e filtrar	Pets: `/pets`
`GET /api/pets/{id}`	Consultar perfil	Pets: `/pets/{id}`
`GET/PUT /api/preferences`	Consultar / guardar quiz	Preferências: `/preferences/{visitor_id}`
`GET /api/favorites`	Listar favoritos	Preferências: `/favorites/{visitor_id}`
`PUT/DELETE /api/favorites/{id}`	Adicionar / remover	Preferências: `/favorites/{visitor_id}/{pet_id}`
`GET /api/recommendations`	Pontuar e ordenar	Recomendações: `/recommendations/{visitor_id}`
`GET /api/status`	Health checks	Gateway e serviços internos

As rotas dos serviços também respondem diretamente nas portas locais `8001`, `8002` e `8003`. Todos oferecem `/docs` e `/openapi.json`. FastAPI documenta a geração automática de documentação interativa e o suporte a OpenAPI [2].

5 Funcionamento passo a passo

5.1 Página inicial e transparência

O cabeçalho oferece navegação para catálogo, match e favoritos. O hero explica a proposta. O aviso acadêmico deixa visível que todos os dados são exemplos. Em celular, a composição empilha textos e ilustração sem esconder o CTA.



Figura 4 — Abertura em viewport móvel; a captura integral desktop com o painel de serviços está em `docs/screenshots/patinhas-desktop.png`.

5.2 Questionário

A pessoa seleciona espécie, porte, tipo de moradia e nível de atividade, além de poder informar se há crianças. Não há conta. Ao enviar, as respostas são salvas no contexto demonstrativo `demo`, e o navegador pede recomendações ao Gateway.

5.3 Recomendações

Os cartões exibem os pets ordenados, a estimativa percentual, um rótulo e motivos por critério. A figura mostra o resultado do questionário processado na interface do protótipo. Para a apresentação, é possível preencher e reenviar o formulário com outras opções.

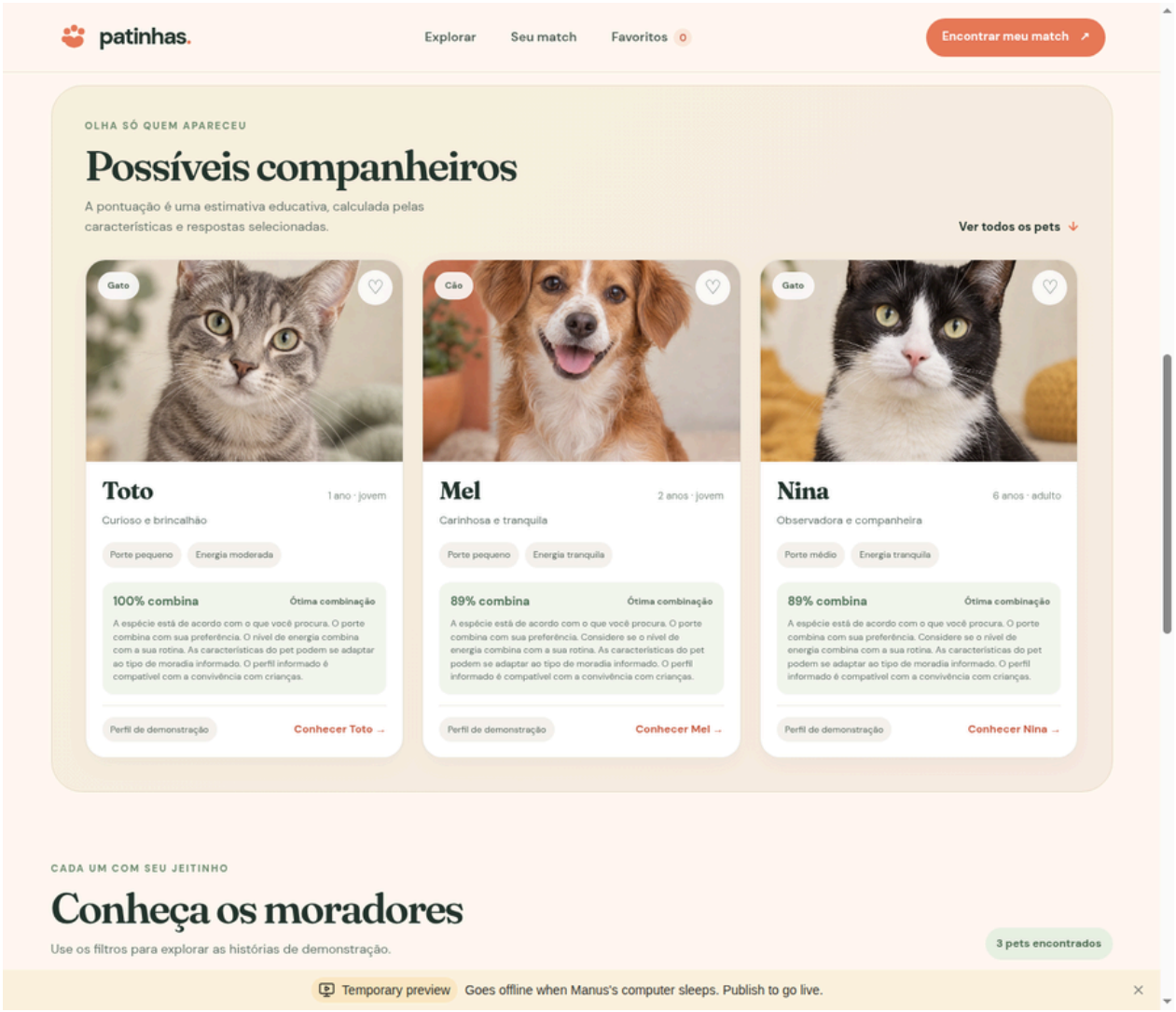


Figura 5 — Recomendações demonstrativas com pontuação e justificativas visíveis.

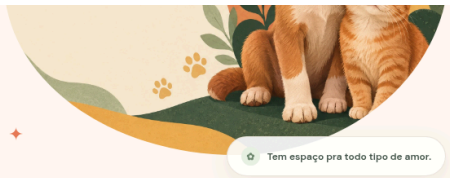
5.4 Catálogo, filtros e perfil

O catálogo permite comparar espécie, idade, temperamento, porte e energia. Os filtros de espécie, porte, idade e energia mudam os resultados sem exigir busca por nome. “Conhecer” abre um perfil com história e características de convivência. O ícone de coração atualiza o estado de favorito.

Responda umas perguntinhas, conheça os pets e descubra quem combina com o seu dia a dia.

Fazer meu match → Quero só dar uma olhadinha ↓

♥ Um bom encontro começa entendendo os dois lados.



✿ Tem espaço pra todo tipo de amor.

ⓘ **Protótipo acadêmico:** todos os pets, fotos e dados são fictícios. Não são anúncios reais nem representam abrigos.

UM QUIZ CURTINHO, UM COMEÇO DE CONVERSA

Como é a vida que vocês vão dividir?

Não existe resposta certa. Só queremos entender um pouquinho da sua rotina.



SEU JEITINHO — leva menos de 1 min

Que tipo de companhia você imagina?

Estou de coração aberto

Porte que combina com seu espaço

Qualquer porte

Onde vocês vão morar?

Apartamento

Seu ritmo para passeios e brincadeiras

Um pouco de cada

☐ Tem crianças na família

Ver quem combina comigo →

Sem cadastro. Suas respostas ficam só neste protótipo de demonstração.

CADA UM COM SEU JEITINHO

Conheça os moradores

Use os filtros para explorar as histórias de demonstração.

3 pets encontrados

Espécie: Todas | Porte: Todos | Idade: Todas | Energia: Qualquer ritmo | Limpar



Cão ♡

Mel — 2 anos · jovem

Carinhosa e tranquila

Porte pequeno · Energia tranquila

Perfil de demonstração — Conhecer Mel →



Gato ♡

Nina — 6 anos · adulto

Observadora e companheira

Porte médio · Energia tranquila

Perfil de demonstração — Conhecer Nina →



Gato ♡

Toto — 1 ano · jovem

Curioso e brincalhão

Porte pequeno · Energia moderada

Perfil de demonstração — Conhecer Toto →

GUARDADOS COM CARINHO

Seus favoritos

Volte aqui para rever os pets que chamaram sua atenção. ♡

Seu mural está vazio por enquanto. Toque no coração de um pet para guardar.

POR TRÁS DAS PATINHAS

Uma equipe de serviços

Figura 6 — Captura integral desktop: abertura, quiz, catálogo, mural vazio e status dos serviços.

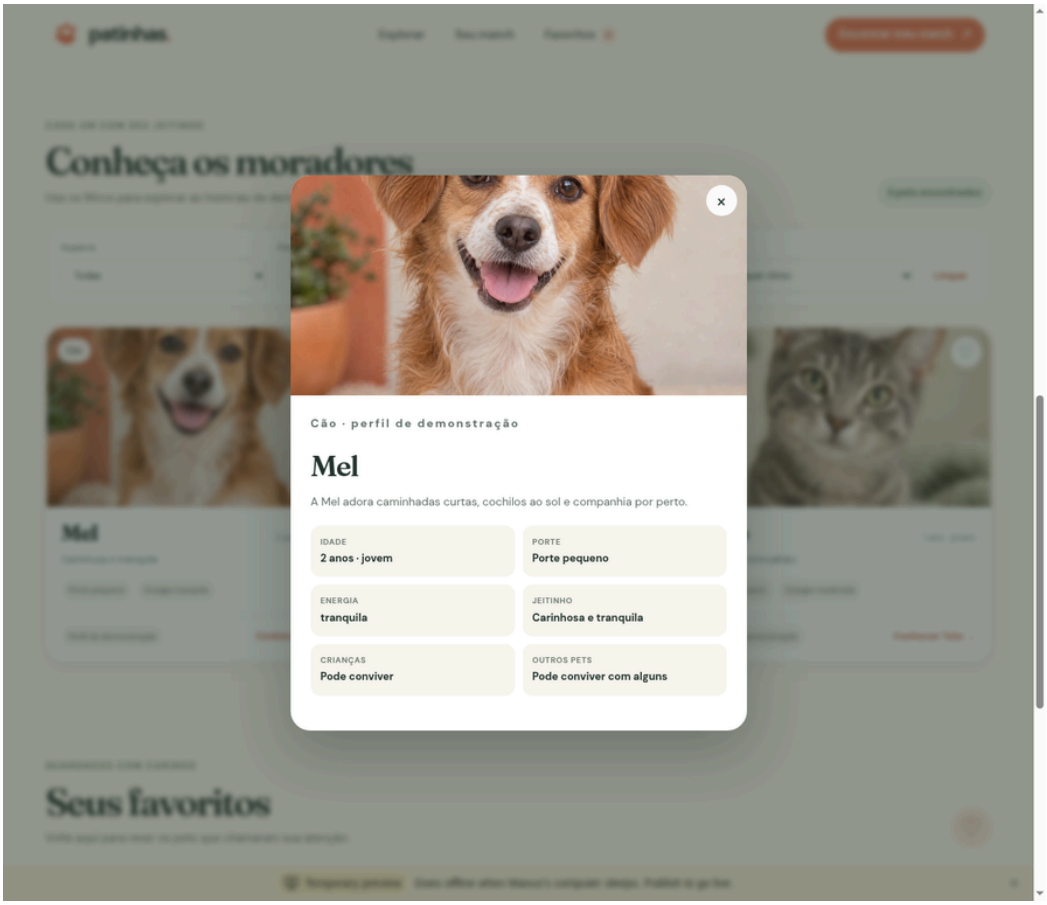


Figura 7 — Modal de perfil demonstrativo aberto a partir do cartão da Mel.

5.5 Favoritos

O coração faz uma chamada `PUT /api/favorites/{id}`; a remoção chama `DELETE`. A interface atualiza o contador e o mural. A persistência é mantida no SQLite próprio do serviço Preferências/Favoritos. O comportamento foi exercitado no Preview: adição, atualização após recarregar a página e remoção do dado de teste.

5.6 Estado e documentação

Ao final da página, a área "Por trás das Patinhas" lista Gateway e microserviços em execução ou indisponíveis. Os links abrem Swagger UI do serviço correspondente. No desenvolvimento local, use `http://127.0.0.1:3000/docs` para o Gateway e `http://127.0.0.1:8001/docs`, `:8002/docs` e `:8003/docs` para os serviços.

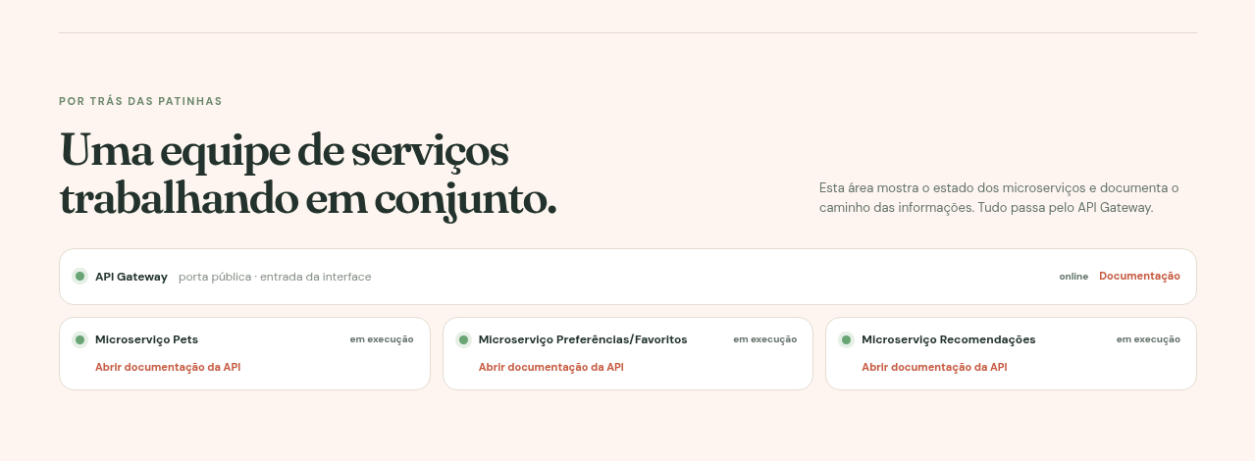


Figura 8 — Interface principal destacando o Gateway e os três microserviços em execução.

6 Organização, testes e estado atual

O repositório organiza cada responsabilidade: `gateway/main.py`; `services/pets/`; `services/preferences/`; `services/recommendations/`; interface em `web/`; inicializador em `run.py`; testes em `tests/`; documentação e diagramas em `docs/`. Nenhum microserviço acessa diretamente o banco de outro. A suíte cobre oito cenários: três verificam pontuação e justificativas do recomendador; os demais exercitam filtros e 404 do catálogo, valores padrão e persistência de preferências/favoritos após reinício, além do roteamento do Gateway. Na checagem de 27/10/2026, os oito testes passaram; também passaram a compilação de módulos Python, verificação sintática JavaScript e leitura dos JSON de design e rotas. A interface foi observada em desktop e celular, e o fluxo principal do quiz e favoritos foi operado no navegador.

6.1 Limites e melhorias futuras

A entrega inicial prioriza funcionamento e modelagem compreensível. O usuário comum demonstrativo é sempre `demo`. Não há login, chat, integração com abrigos, busca geográfica nem anúncios reais. O recomendador é um conjunto de regras, não um sistema clínico ou decisor de adoção. Somente após cumprir o escopo base podem ser avaliados melhorias como autenticação ou chat.

7 Reuso demonstrado e referências visuais

O levantamento de inspiração usou quatro páginas. Petfinder inspira entradas separadas para cães e gatos, navegação de busca e favoritos [5]. Adopt a Pet explica filtros como porte, idade e convivência, os quais foram reduzidos a um conjunto pequeno para este protótipo [6]. A página Troakly no Dribbble foi observada como referência geral para hero emocional e paleta quente [7]. O case PawHome da HALO LAB serviu como referência de linguagem amigável, hierarquia simples e foco nas necessidades da pessoa usuária [8].

Essas fontes orientam padrões gerais. O JSON `design/design-system.json` é uma síntese própria, não uma extração automática nem cópia de código, layout, logotipo ou imagens de terceiros. As imagens do projeto são originais e os nomes/fotos são fictícios.

8 Glossário

componente de software – **componente de software**: Unidade reutilizável que encapsula uma responsabilidade e pode ser aplicada em mais de uma parte da interface.

FastAPI – **FastAPI**: Framework Python escolhido para declarar e servir as APIs HTTP.

API Gateway – **API Gateway**: Ponto de entrada que encaminha solicitações da interface às APIs internas.

microserviço – **microserviço**: Serviço independente, com responsabilidade delimitada e comunicação por interface de rede.

OpenAPI – **OpenAPI**: Formato de descrição das operações de uma API HTTP, usado para gerar documentação interativa.

Scrum – **Scrum**: Referencial de trabalho ágil usado aqui de forma simplificada, com backlog ordenado e ciclos curtos de entrega.

SQLite – **SQLite**: Mecanismo relacional leve usado como banco local independente de cada serviço.

9 Referências

[1] Python Software Foundation. **Python 3 Documentation**. Disponível em: docs.python.org/3. Acesso em 27 out. 2026.

[2] Sebastián Ramírez e colaboradores. **FastAPI Documentation**. Disponível em: fastapi.tiangolo.com. Acesso em 27 out. 2026.

[3] SQLite Consortium. **SQLite Documentation**. Disponível em: sqlite.org/docs.html. Acesso em 27 out. 2026.

[4] MDN Web Docs. **Using custom elements**. Disponível em: developer.mozilla.org. Acesso em 27 out. 2026.

[5] Petfinder. **Find a Pet**. Disponível em: petfinder.com. Acesso em 27 out. 2026.

[6] Frances Vega. **How Filters Make Pet Searching Easier (and Faster)**. Adopt a Pet, 28 jan. 2026. Disponível em: adoptapet.com. Acesso em 27 out. 2026.

[7] Troakly. **Dog Rescue & Adoption Landing Page Design UI for Animal Shelters**. Dribbble. Disponível em: dribbble.com. Acesso em 27 out. 2026.

[8] HALO LAB. **PawHome — Pet Adoption Platform UX/UI Branding**. Behance, 20 maio 2025. Disponível em: behance.net. Acesso em 27 out. 2026.

10 Conferência para a entrega final

Antes da entrega de 9 de novembro de 2026, complete a identificação da capa e substitua o texto provisório do campo GitHub pelo endereço do repositório confirmado. O professor pede que o projeto completo esteja funcionando no GitHub e que a capa do relatório traga o link correspondente.

No momento desta versão de trabalho, o protótipo e a documentação estão prontos para essa transferência; a seleção e autorização do repositório ainda dependem da conta GitHub do estudante. Depois do envio, confirme que o README permite instalar, iniciar e demonstrar a aplicação e que o link da capa abre o código esperado.